

PROGRAMACIÓN II

TP 8: Interfaces y Excepciones en Java

Alumno: Chiavón, Cristian

Repositorio GitHub: <https://github.com/cridvon/UTN-TUPaD-P2>

Caso Practico

Parte 1: Interfaces en un sistema de E-commerce

1. Crear una interfaz **Pagable** con el método **calcularTotal()**.

```
package ecommerce;

public interface Pagable {
    double calcularTotal();
}
```

2. Clase **Producto**: tiene nombre y precio, implementa **Pagable**.

```
package ecommerce;

public class Producto implements Pagable {
    private String nombre;
    private double precio;

    public Producto(String nombre, double precio) {
        this.nombre = nombre;
        this.precio = precio;
    }

    @Override
    public double calcularTotal() {
        return precio;
    }

    @Override
    public String toString() {
        return nombre + " - $" + precio;
    }
}
```

3. Clase **Pedido**: tiene una lista de productos, implementa **Pagable** y calcula el total del pedido.

```
package ecommerce;

import java.util.ArrayList;
import java.util.List;

public class Pedido implements Pagable {
    private List<Producto> productos;
    private String estado;
    private Cliente cliente;

    public Pedido(Cliente cliente) {
        this.cliente = cliente;
        this.productos = new ArrayList<>();
        this.estado = "Pendiente";
    }

    public void agregarProducto(Producto p) {
        productos.add(p);
    }

    @Override
    public double calcularTotal() {
        double total = 0;
        for (Producto p : productos) {
            total += p.calcularTotal();
        }
        return total;
    }

    public void cambiarEstado(String nuevoEstado) {
        this.estado = nuevoEstado;
        cliente.notificarCambio("El pedido cambió su estado a: " + nuevoEstado);
    }

    public String getEstado() {
        return estado;
    }
}
```

4. Ampliar con interfaces **Pago** y **PagoConDescuento** para distintos medios de pago (**TarjetaCredito**, **PayPal**), con métodos **procesarPago(double)** y **aplicarDescuento(double)**.

```
package ecommerce;

public interface Pago {
    void procesarPago(double monto);
}

PagoConDescuento.java X
package ecommerce;

public interface PagoConDescuento extends Pago {
    double aplicarDescuento(double monto);
}

TarjetaCredito.java X
package ecommerce;

public class TarjetaCredito implements PagoConDescuento {

    @Override
    public double aplicarDescuento(double monto) {
        // 10% de descuento
        return monto * 0.9;
    }

    @Override
    public void procesarPago(double monto) {
        System.out.println("Pago con tarjeta procesado por $" + monto);
    }
}

PayPal.java X
package ecommerce;

public class PayPal implements Pago {

    @Override
    public void procesarPago(double monto) {
        System.out.println("Pago con PayPal procesado por $" + monto);
    }
}
```

5. Crear una interfaz **Notificable** para notificar cambios de estado. La clase **Cliente** implementa dicha interfaz y **Pedido** debe notificarlo al cambiar de estado.

```
Notificable.java x
Source History
1 package ecommerce;
2
3 public interface Notificable {
4     void notificarCambio(String mensaje);
5 }
6

Cliente.java x
Source History
1 package ecommerce;
2
3 public class Cliente implements Notificable {
4     private String nombre;
5     private String email;
6
7     public Cliente(String nombre, String email) {
8         this.nombre = nombre;
9         this.email = email;
10    }
11
12    @Override
13    public void notificarCambio(String mensaje) {
14        System.out.println("Notificación para " + nombre + ": " + mensaje);
15    }
16
17    @Override
18    public String toString() {
19        return nombre + " (" + email + ")";
20    }
21 }

Pedido.java x
Source History
27     return total;
28 }
29
30 public void cambiarEstado(String nuevoEstado) {
31     this.estado = nuevoEstado;
32     cliente.notificarCambio("El pedido cambió su estado a: " + nuevoEstado);
33 }
34
35 public String getEstado() {
36     return estado;
37 }
38 }
```

Clase Main y ejecución:

```
package ecommerce;

public class Main {

    public static void main(String[] args) {

        Cliente cliente = new Cliente("Cristian", "cristian@mail.com");

        Producto p1 = new Producto("Mouse", 1500);
        Producto p2 = new Producto("Teclado", 2500);

        Pedido pedido = new Pedido(cliente);
        pedido.agregarProducto(p1);
        pedido.agregarProducto(p2);

        System.out.println("Total del pedido: $" + pedido.calcularTotal());

        // Pago con tarjeta (con descuento)
        TarjetaCredito tarjeta = new TarjetaCredito();
        double montoConDescuento = tarjeta.aplicarDescuento(pedido.calcularTotal());
        tarjeta.procesarPago(montoConDescuento);

        // Cambiar estado del pedido
        pedido.cambiarEstado("Enviado");
    }
}

it - EcommerceInterfaces (run)

run:
Total del pedido: $4000.0
Pago con tarjeta procesado por $3600.0
Notificación para Cristian: El pedido cambi su estado a: Enviado
BUILD SUCCESSFUL (total time: 1 second)
```

Parte 2: Ejercicios sobre Excepciones

1. División segura

- Solicitar dos números y dividirlos. Manejar **ArithmeticException** si el divisor es cero.

```
package excepciones;

import java.util.Scanner;

public class DivisionSegura {

    public static void dividir() {
        Scanner sc = new Scanner(System.in);

        try {
            System.out.print("Ingrese el numerador: ");
            int numerador = sc.nextInt();

            System.out.print("Ingrese el denominador: ");
            int denominador = sc.nextInt();

            int resultado = numerador / denominador;
            System.out.println("Resultado: " + resultado);

        } catch (ArithmeticException e) {
            System.out.println("Error: No se puede dividir por cero.");
        }
    }
}
```

2. Conversión de cadena a número

- Leer texto del usuario e intentar convertirlo a `int`. Manejar `NumberFormatException` si no es válido.

```
package excepciones;

import java.util.Scanner;

public class ConversionCadenaNumero {

    public static void convertir() {
        Scanner sc = new Scanner(System.in);

        try {
            System.out.print("Ingrese un número entero: ");
            String texto = sc.nextLine();

            int numero = Integer.parseInt(texto);
            System.out.println("Número convertido: " + numero);

        } catch (NumberFormatException e) {
            System.out.println("Error: el texto ingresado no es un número válido.");
        }
    }
}
```

3. Lectura de archivo

- Leer un archivo de texto y mostrarlo. Manejar `FileNotFoundException` si el archivo no existe.

```
package excepciones;

import java.io.File;
import java.io.FileNotFoundException;
import java.util.Scanner;

public class LecturaArchivo {

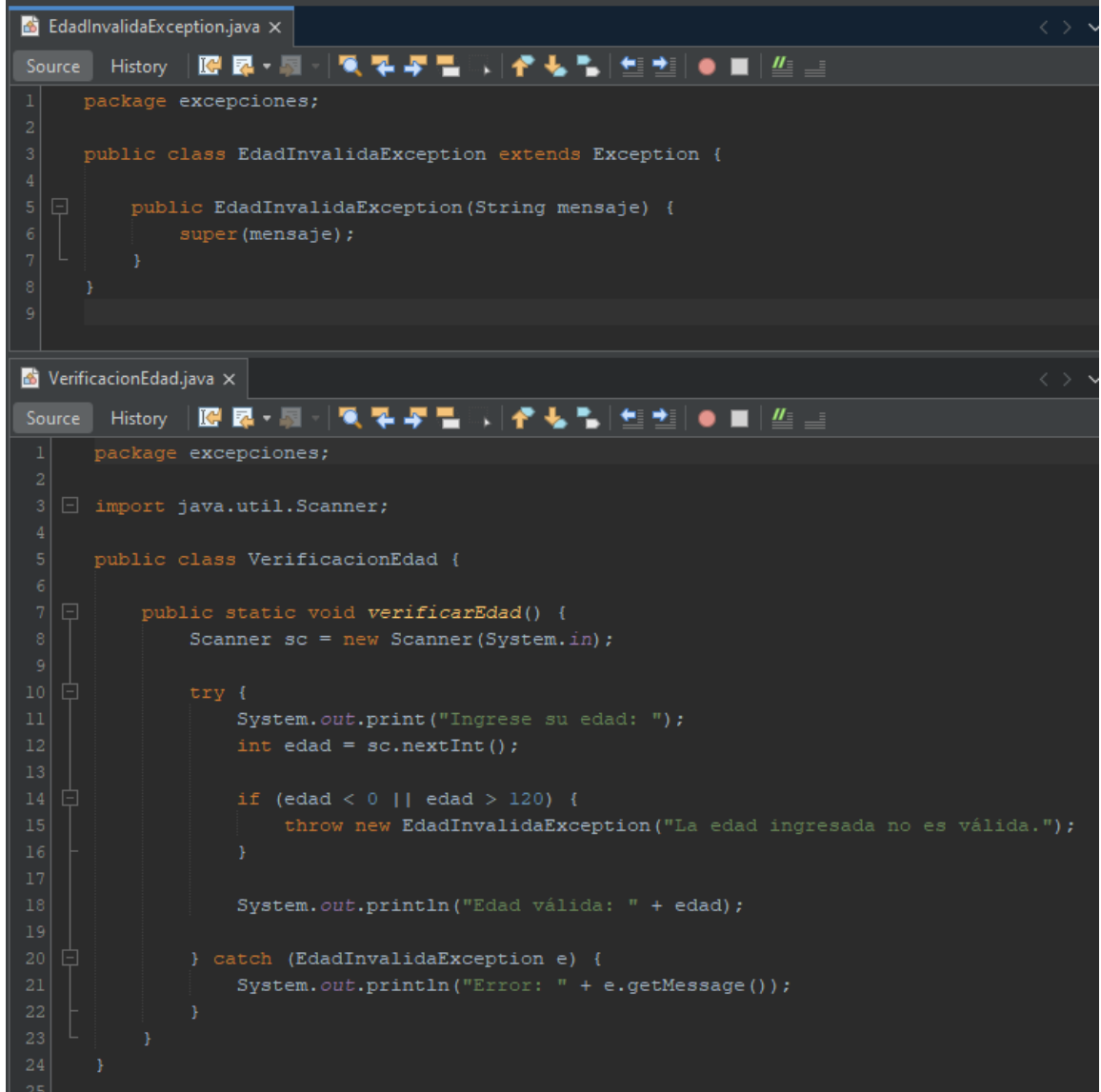
    public static void leerArchivo(String ruta) {
        try {
            File archivo = new File(ruta);
            Scanner sc = new Scanner(archivo);

            System.out.println("Contenido del archivo:");
            while (sc.hasNextLine()) {
                System.out.println(sc.nextLine());
            }
            sc.close();

        } catch (FileNotFoundException e) {
            System.out.println("Error: el archivo no existe o la ruta es incorrecta.");
        }
    }
}
```

4. Excepción personalizada

- Crear **EdadInvalidaException**. Lanzarla si la edad es menor a 0 o mayor a 120. Capturarla y mostrar mensaje.



```
1 package excepciones;
2
3 public class EdadInvalidaException extends Exception {
4
5     public EdadInvalidaException(String mensaje) {
6         super(mensaje);
7     }
8 }
9
10
11
12
13
14
15
16
17
18
19
20
21
22
23
24
25
```

```
1 package excepciones;
2
3 import java.util.Scanner;
4
5 public class VerificacionEdad {
6
7     public static void verificarEdad() {
8         Scanner sc = new Scanner(System.in);
9
10        try {
11            System.out.print("Ingrese su edad: ");
12            int edad = sc.nextInt();
13
14            if (edad < 0 || edad > 120) {
15                throw new EdadInvalidaException("La edad ingresada no es válida.");
16            }
17
18            System.out.println("Edad válida: " + edad);
19
20        } catch (EdadInvalidaException e) {
21            System.out.println("Error: " + e.getMessage());
22        }
23    }
24 }
25
```


5. Uso de try-with-resources

- Leer un archivo con **BufferedReader** usando **try-with-resources**.

Manejar **IOException** correctamente.

```
package excepciones;

import java.io.BufferedReader;
import java.io.FileReader;
import java.io.IOException;

public class LecturaConTryWithResources {

    public static void leerArchivo(String ruta) {
        try (BufferedReader br = new BufferedReader(new FileReader(ruta))) {
            System.out.println("Leyendo archivo con try-with-resources:");
            String linea;
            while ((linea = br.readLine()) != null) {
                System.out.println(linea);
            }
        } catch (IOException e) {
            System.out.println("Error al leer el archivo: " + e.getMessage());
        }
    }
}
```

Clase Main

```
package excepciones;

public class Main {

    public static void main(String[] args) {

        System.out.println("=== 1. División segura ===");
        DivisionSegura.dividir();

        System.out.println("\n=== 2. Conversión de cadena a número ===");
        ConversionCadenaNumero.convertir();

        System.out.println("\n=== 3. Lectura de archivo ===");
        LecturaArchivo.leerArchivo("archivo.txt"); // cambiar la ruta según tu archivo

        System.out.println("\n=== 4. Verificación de edad ===");
        VerificacionEdad.verificarEdad();

        System.out.println("\n=== 5. Lectura con try-with-resources ===");
        LecturaConTryWithResources.leerArchivo("archivo.txt");
    }
}
```

Ejecución con datos válidos.

```
Output - Excepciones (run)

run:
=== 1. División segura ===
Ingrese el numerador: 10
Ingrese el denominador: 2
Resultado: 5

=== 2. Conversión de cadena a número ===
Ingrese un número entero: 12345
Número convertido: 12345

=== 3. Lectura de archivo ===
Contenido del archivo:
Hoy Es Jueves-

=== 4. Verificación de edad ===
Ingrese su edad: 56
Edad válida: 56

=== 5. Lectura con try-with-resources ===
Leyendo archivo con try-with-resources:
Hoy Es Jueves-
BUILD SUCCESSFUL (total time: 12 seconds)
```

Ejecución con datos inválidos.

```
Output - Excepciones (run)

run:
=== 1. División segura ===
Ingrese el numerador: 10
Ingrese el denominador: 0
Error: No se puede dividir por cero.

=== 2. Conversión de cadena a número ===
Ingrese un número entero: qwer
Error: el texto ingresado no es un número válido.

=== 3. Lectura de archivo ===
Error: el archivo no existe o la ruta es incorrecta.

=== 4. Verificación de edad ===
Ingrese su edad: 150
Error: La edad ingresada no es válida.

=== 5. Lectura con try-with-resources ===
Error al leer el archivo: archivo.txt (El sistema no puede encontrar el archivo especificado)
BUILD SUCCESSFUL (total time: 16 seconds)
```